COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	07/16/2026	**DATE SUBMITTED:**	07/21/2026
NAME:	Cabug, Jeffordniel H.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS- ON ACTIVITY NO. 2.1 **[Arrays, Pointers and Dynamic Memory Allocation]**

1. PROCEDURE OUTPUT

ILO A:

Screenshot

CODE:

```cpp
#include <iostream>
#include <string>

class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        //constructor
        Student(std::string newName ="JEP", int newAge = 18){
            studentName = newName;
            studentAge = newAge;
            std::cout << "Constructor Called. " << std::endl;
        };

        //deconstructor
        ~Student(){
            std::cout << "Destructor Called. " << std::endl;
        }

        //copy constructor
        Student (const Student &copyStudent){
            std::cout << "Copy Constructor Called." << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }

        //Display Attributes
        void printDetails(){
            std::cout<< this->studentName << " " << this->studentAge << std::endl;
        }
};

int main (){

    Student student1("Roman", 28);
    Student student2(student1);
    Student student3;
    student3 = student2;


    return 0;
}
```

	OUTPUT:
	`Constructor Called.`
	`Copy Constructor Called.`
	`Constructor Called.`
	`Destructor Called.`
	`Destructor Called.`
	`Destructor Called.`
	
	`--------------------------------`
	`Process exited after 0.01518 seconds with return value 0`
	`Press any key to continue . . .`

Observation	We have 3 objects in int main, since the object “student1” matches the constructors parameter that's why it triggers the constructor and then prints out the “Constructor Called”. The second object which is the “student2” has the constructor argument (student1). It creates a brand new object using an existing object (student1), it matches the copy constructor (const Student &copyStudent) that's why it prints out the “Copy Constructor Called”. Since the student 3 is empty it calls the values that were created **(std::string newName ="JEP", int newAge = 18) default** and then matches the constructor parameter, that’s why it prints out the “Constructor Called”. Then the student3 = student2 is like deleting inside of the object student 3 to 1 and then prints "Destructor called” that's why there’s a 3 output and then it exits the program.

Initial Driver Program

Screenshot	**CODE:**

```cpp
#include <iostream>
#include <string>

class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        //constructor
        Student(std::string newName ="JEP", int newAge = 18){
            studentName = newName;
            studentAge = newAge;
            std::cout << "Constructor Called. " << std::endl;
        };

        //deconstructor
        ~Student(){
            std::cout << "Destructor Called. " << std::endl;
        }

        //copy constructor
        Student (const Student &copyStudent){
            std::cout << "Copy Constructor Called." << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }

        //Display Attributes
        void printDetails(){
            std::cout<< this->studentName << " " << this->studentAge << std::endl;
        }
};

int main() {
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

return 0;
}
```

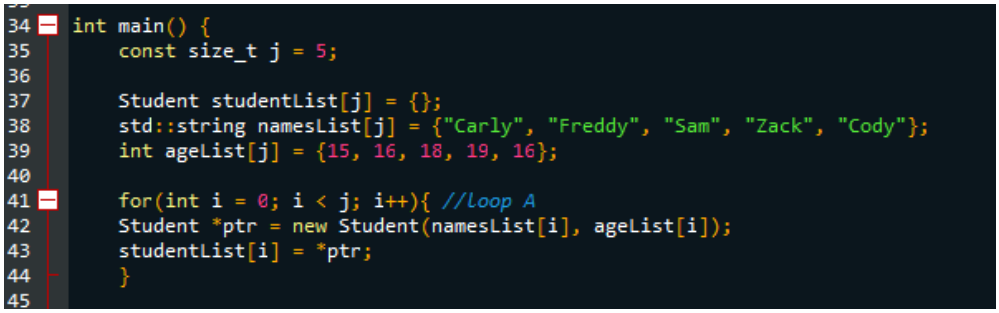
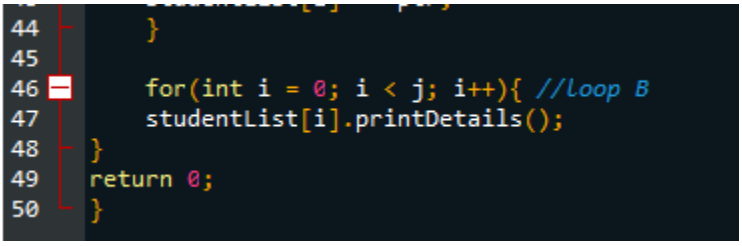
OUTPUT:

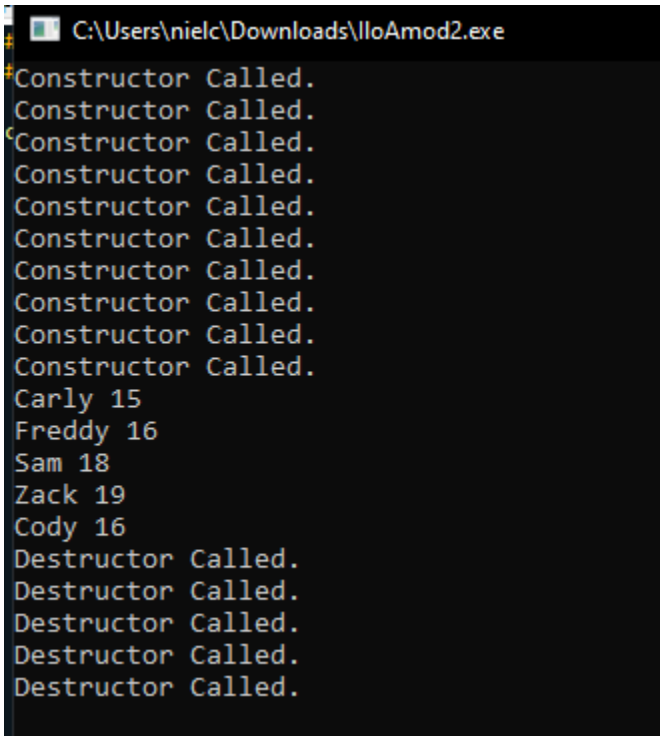
```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.

--------------------------------
Process exited after 0.0107 seconds with return value 0
Press any key to continue . . .
```

Observation	It uses array sets to 5 from index 0 to 4 to initialize each slot, since the braces are empty it triggers a loop constructor call. The studentlist is separate and has no connection so it sits there. Since no data was provided it uses the default parameters **name JEP** and **age 18** into slot 0 then prints “Constructor Called”. Then a new slot was created slot 1it will do the same as slot 0, and it goes on until slot 4. When the code hits return 0; the array must be wiped out, It empties and deletes slot 0-4 and prints out the “Destructor Called” and then exits the program.

Modified Driver Program with Student Lists

Loop A	
Observation	An array of five Student objects is created. Since no values are given, the default constructor is automatically called five times. Each object is created with the default values, which are "JEP" for the name and 18 for the age. Inside Loop A, the “new” creates a new Student object in the heap during each of the five loops. The parameterized constructor is called using the values from namesList and ageList. After that, the data from the heap object is copied into the matching object in the stack array. This uses the default assignment operator to copy the data. After the loops, the pointer variable ptr is removed because it goes out of scope. However, the heap memory is not deleted because there is no delete ptr; statement. The program can no longer access the five objects that were created in the heap.
Loop B	
Observation	Loop B uses a loop counter “i” to go through each element of the studentList array, starting from index 0 to 4. In each loop, the program accesses the Student object stored at studentList[i] and calls its printDetails() function. The function then reads the student's name and age stored in the object and displays them on the screen using “std::cout”. Since the objects were updated earlier with data from namesList and ageList, the output shows the new values instead of the default ones.

Output	
Observation	The"Constructor Called." is printed ten times. This happens because the studentList array creates five Student objects using the default constructor which is the “JEP” “18”, and Loop A creates another five Student objects on the heap using the new keyword. The data from the heap objects is then copied into the objects inside the studentList array, so when Loop B runs, it prints the correct student names and ages instead of the default values. At the end of the program, when return 0; , the message "Destructor Called." is printed five times because the studentList array is automatically destroyed. The five objects created on the heap are never deleted because there is no delete statement. This causes a memory leak since the program leaves those objects in memory until it exits the program.

Final Driver Program

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve programming problems using dynamic memory allocation, arrays and pointers
Jenna’s Grocery List

Jenna wants to buy the following fruits and vegetables for her daily consumption. However, she needs to distinguish between fruit and vegetable, as well as calculate the sum of prices that she has to pay in total.

Problem 1: Create a class for the fruit and the vegetable classes. Each class must have a constructor, deconstructor, copy constructor and copy assignment operator. They must also have all relevant attributes (such as name, price and quantity) and functions (such as calculate sum) as presented in the problem

description above.

Problem 2: Create an array GroceryList in the driver code that will contain all items in Jenna's Grocery List. You must then access each saved instance and display all details about the items.

Problem 3: Create a function TotalSum that will calculate the sum of all objects listed in Jenna's Grocery List.

Problem 4: Delete the Lettuce from Jenna's GroceryList list and de-allocate the memory assigned.

CODE:

```cpp
#include <iostream>
#include <string>
using namespace std;

class Grocery {
protected:
    string name;
    double price;
    int quantity;

public:
    // Default Constructor
    Grocery() {
        name = "";
        price = 0;
        quantity = 0;
        cout << "GROCERY Constructor was called." << endl;
    }

    // Parameterized Constructor
    Grocery(string n, double p, int q) {
        name = n;
        price = p;
        quantity = q;
        cout << "GROCERY Constructor was called." << endl;
    }

    // Copy Constructor
    Grocery(const Grocery& other) {
        name = other.name;
        price = other.price;
        quantity = other.quantity;
    }

    // Copy Assignment Operator
    Grocery& operator=(const Grocery& other) {
        if (this != &other) {
            name = other.name;
            price = other.price;
            quantity = other.quantity;
        }
        return *this;
    }

    virtual double calculateSum() const {
        return price * quantity;
    }

    virtual void display() const = 0;

    virtual ~Grocery() {
        cout << "GROCERY Destructor was called." << endl;
    }
};

class Fruit : public Grocery {
public:
    Fruit(string n, double p, int q)
        : Grocery(n, p, q) {
        cout << "FRUIT Constructor was called." << endl;
    }

```

```cpp
    Fruit(const Fruit& other)
        : Grocery(other) {}

    Fruit& operator=(const Fruit& other) {
        Grocery::operator=(other);
        return *this;
    }

    void display() const override {
        cout << "Fruit: " << name << endl;
        cout << "Price: PHP " << price << endl;
        cout << "Quantity: " << quantity << endl;
        cout << "Subtotal: PHP " << calculateSum() << endl << endl;
    }

    ~Fruit() {
        cout << "FRUIT Destructor was called." << endl;
    }
};

class Vegetable : public Grocery {
public:
    Vegetable(string n, double p, int q)
        : Grocery(n, p, q) {
        cout << "VEG Constructor was called." << endl;
    }

    Vegetable(const Vegetable& other)
        : Grocery(other) {}

    Vegetable& operator=(const Vegetable& other) {
        Grocery::operator=(other);
        return *this;
    }

    void display() const override {
        cout << "Vegetable: " << name << endl;
        cout << "Price: PHP " << price << endl;
        cout << "Quantity: " << quantity << endl;
        cout << "Subtotal: PHP " << calculateSum() << endl << endl;
    }

    ~Vegetable() {
        cout << "VEG Destructor was called." << endl;
    }
};

double TotalSum(Grocery* list[], int size) {
    double total = 0;

    for (int i = 0; i < size; i++) {
        if (list[i] != nullptr)
            total += list[i]->calculateSum();
    }

    return total;
}

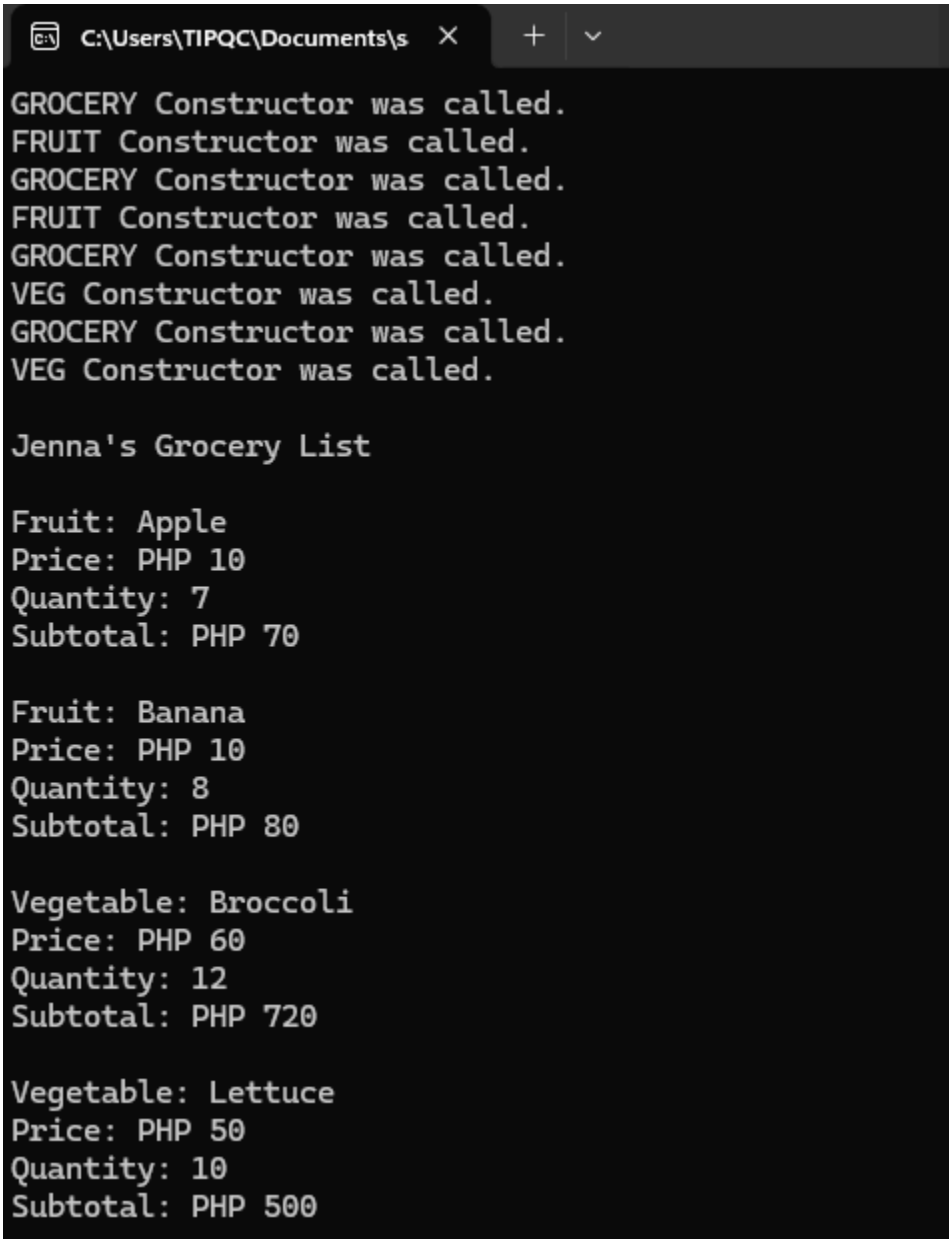
int main() {

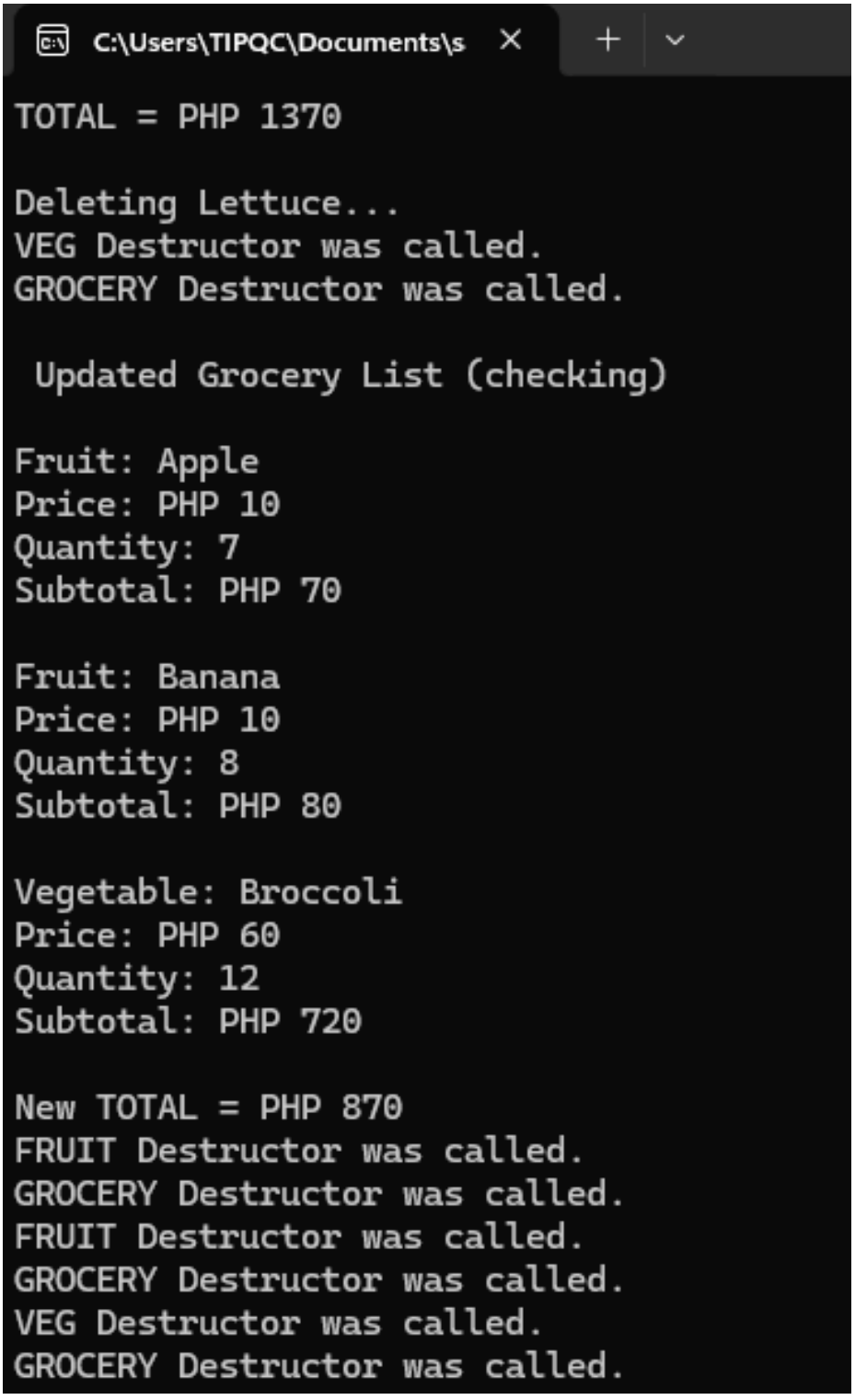
    Grocery* GroceryList[4];

```

```
125         GroceryList[0] = new Fruit("Apple", 10, 7);
126         GroceryList[1] = new Fruit("Banana", 10, 8);
127         GroceryList[2] = new Vegetable("Broccoli", 60, 12);
128         GroceryList[3] = new Vegetable("Lettuce", 50, 10);
129
130         cout << "\nJenna's Grocery List \n\n";
131
132         for (int i = 0; i < 4; i++)
133             GroceryList[i]->display();
134
135         cout << "TOTAL = PHP " << TotalSum(GroceryList, 4) << endl;
136
137         cout << "\nDeleting Lettuce..." << endl;
138
139         delete GroceryList[3];
140         GroceryList[3] = nullptr;
141
142         cout << "\n Updated Grocery List (checking) \n\n";
143
144         for (int i = 0; i < 4; i++) {
145             if (GroceryList[i] != nullptr)
146                 GroceryList[i]->display();
147         }
148
149         cout << "New TOTAL = PHP " << TotalSum(GroceryList, 4) << endl;
150
151         //
152         for (int i = 0; i < 3; i++) {
153             delete GroceryList[i];
154             GroceryList[i] = nullptr;
155         }
156
157         return 0;
158     }
```

OUTPUT:


C:\Users\TIPQC\Documents\s
GROCERY Constructor was called.
FRUIT Constructor was called.
GROCERY Constructor was called.
FRUIT Constructor was called.
GROCERY Constructor was called.
VEG Constructor was called.
GROCERY Constructor was called.
VEG Constructor was called.
Jenna's Grocery List
Fruit: Apple
Price: PHP 10
Quantity: 7
Subtotal: PHP 70
Fruit: Banana
Price: PHP 10
Quantity: 8
Subtotal: PHP 80
Vegetable: Broccoli
Price: PHP 60
Quantity: 12
Subtotal: PHP 720
Vegetable: Lettuce
Price: PHP 50
Quantity: 10
Subtotal: PHP 500


C:\Users\TIPQC\Documents\s
TOTAL = PHP 1370
Deleting Lettuce...
VEG Destructor was called.
GROCERY Destructor was called.
Updated Grocery List (checking)
Fruit: Apple
Price: PHP 10
Quantity: 7
Subtotal: PHP 70
Fruit: Banana
Price: PHP 10
Quantity: 8
Subtotal: PHP 80
Vegetable: Broccoli
Price: PHP 60
Quantity: 12
Subtotal: PHP 720
New TOTAL = PHP 870
FRUIT Destructor was called.
GROCERY Destructor was called.
FRUIT Destructor was called.
GROCERY Destructor was called.
VEG Destructor was called.
GROCERY Destructor was called.

ANALYSIS:

This program is similar to the code we made in the Procedure Output activity because it also uses a complete class with a constructor, destructor, copy constructor, and copy assignment operator. The biggest difference is that this program uses a **GroceryList** array to store all the fruit and vegetable objects in one list. The GroceryList part was the most challenging for me because I was not familiar with how to store different objects in a single array. I had to do some research and learn how pointers and the copy assignment operator work. After understanding them, I was able to store the fruit and vegetable objects in the GroceryList and access them later. I also learned how classes can work together. In this program, the Fruit and Vegetable classes are connected to the Grocery class through inheritance, which helped me organize the code better and avoid repeating the same variables and functions. I added an **Updated Grocery List** after deleting the **Lettuce** object. This allowed me to check if the object was successfully removed from the array. It also helped me make sure that the memory was properly deallocated and that there were no leftover objects or memory leaks.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGpt, Ai overview, Grammar Checker C++ visualizer**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation, fixing grammar**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I use it to explain concepts like the destructor and the constructor and I use a visualizer to know exactly how the program works.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.